
Search Typeahead System

Project Report

Author	Aryan Bhendarkar
Project Type	High-Level System Design
Version	1.0.0
Date	June 2026
Status	Complete

Table of Contents

#	Section
1	[Project Overview](#1-project-overview)
2	[System Architecture](#2-system-architecture)
3	[Dataset & Loading Guide](#3-dataset--loading-guide)
4	[API Documentation](#4-api-documentation)
5	[Design Choices & Trade-offs](#5-design-choices--trade-offs)
6	[Performance Report](#6-performance-report)
7	[Technology Stack](#7-technology-stack)

1. Project Overview

The **Search Typeahead System** is a backend-focused system design project that delivers real-time autocomplete suggestions — just like Google's search bar. When a user starts typing, the system instantly returns the most relevant and trending suggestions.

What This System Does

```
User types "goo" → System returns:
1. google      (score: 98,234)
2. google maps (score: 84,112)
3. google docs (score: 65,432)
...up to 10 results, instantly
```

[OK] System Goals at a Glance

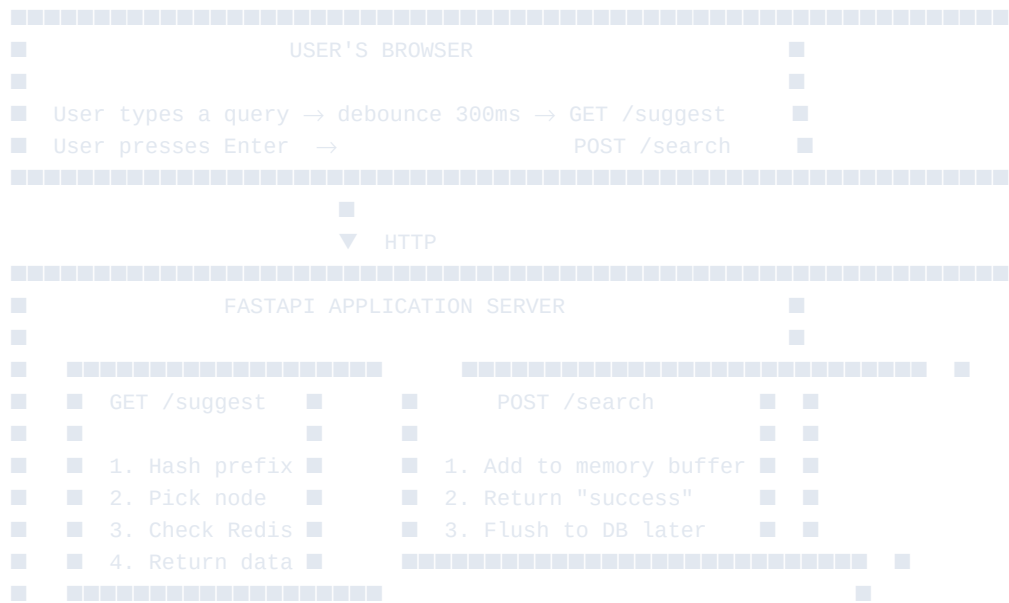
Goal	Target	How It's Achieved
Speed	< 20ms response	Redis cache serves results instantly
Scale	Handles many users	Load spread across 3 Redis nodes
Reliability	Survives node failures	Automatic fallback to other Redis nodes
DB Performance	Reduce write load	Queries buffered & flushed in bulk
Relevant Results	Trending-first ranking	Weighted score: global + weekly + daily

Key Features

- **Real-time Autocomplete** — Suggestions appear as the user types (debounced to avoid flooding the server)
- **Trending Score Ranking** — Results are ranked by popularity (not just alphabetically)
- **Distributed Caching** — Three Redis nodes share the load; each prefix always goes to the same node
- **Write-Behind Buffering** — Searches are collected in memory, then saved to the database in bulk
- **Live Metrics Dashboard** — The UI shows cache hit rate, buffer size, and request counts in real time

2. System Architecture

2.1 High-Level Data Flow





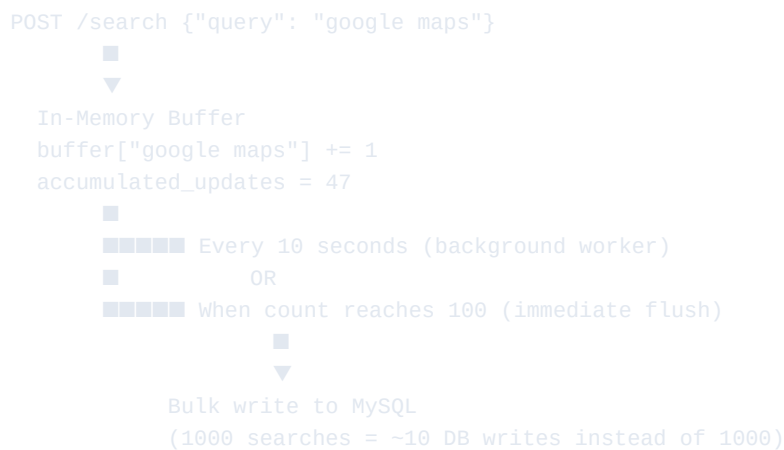
2.2 How a Suggestion Request Works (Step by Step)

Example: User types "goo"

Step	What Happens
1	Frontend sends <code>GET /suggest?q=goo</code> after 300ms debounce
2	Backend strips and lowercases the prefix → "goo"
3	Consistent Hash Ring picks the responsible Redis node (e.g., <code>redis-2</code>)
4	Backend checks Redis for key <code>suggest:goo</code>
5a	Cache HIT → Return stored results immediately (~2–5ms)
5b	Cache MISS → Query MySQL: <code>WHERE query_text LIKE 'goo%' ORDER BY trending_score DESC LIMIT 10</code>
6	Store result back in Redis with 5-minute TTL
7	Return results to user

2.3 How a Search Submission Works

Example: User presses Enter on "google maps"



2.4 Component Breakdown

Consistent Hash Ring

The ring ensures every prefix always routes to the **same Redis node** — no random assignment, no duplication across nodes.

How it works:

- Each Redis node gets 100 virtual positions on a circular ring
- Total: 3 nodes × 100 = 300 positions
- A prefix like "goo" is hashed → placed on the ring
- The nearest clockwise position's node handles it

Ring (circular, 0 → 2^{128}):

...■[redis-1]■[redis-3]■[redis-2]■[redis-1]■[redis-3]■...

↑
"goo" maps here → redis-1 handles it forever

Why 100 virtual nodes? Fewer virtual nodes = uneven load. More virtual nodes = better distribution but more memory. 100 is the sweet spot.

Write-Behind Buffer

The buffer is the main write optimization. Instead of writing to MySQL on every search, the system collects searches in memory and saves them together.

WITHOUT buffer: 1000 searches = 1000 individual MySQL writes ■ Slow
WITH buffer: 1000 searches = ~10 bulk MySQL writes ■ Fast

Buffer flush triggers:

- Every 10 seconds (background timer)
- When buffer accumulates 100+ entries (immediate flush)
- On application shutdown (final drain)

When flushed, what happens in MySQL:

1. Log entries inserted into `query_logs`
2. `global_count` updated for each query
3. `daily_count` and `weekly_count` recalculated from time-window queries
4. `trending_score` recomputed
5. Redis cache refreshed for affected prefixes

Redis Failover

If a Redis node goes down, the system doesn't crash:

```
Primary node fails?
  ■■■ Retry 3 times (10ms → 20ms → 30ms gaps)
    ■
    ■■■ Still fails?
      ■■■ Try next Redis node (2 attempts)
        ■
        ■■■ All nodes fail?
          ■■■ Serve from MySQL directly
              (mark as FAIL in metrics)
```

Database Schema

Table 1: `search_queries` — Main lookup table for autocomplete

Column	Type	Purpose
<code>query_text</code>	VARCHAR(255)	The search term (unique, indexed)
<code>global_count</code>	INT	Total times this was searched (all time)
<code>weekly_count</code>	INT	Searches in the past 7 days
<code>daily_count</code>	INT	Searches in the past 24 hours
<code>trending_score</code>	FLOAT	Composite ranking score

Table 2: `query_logs` — Time-series log for rolling counts

Column	Type	Purpose
<code>query_text</code>	VARCHAR(255)	Which query was searched
<code>query_time</code>	DATETIME	When it was searched

Index Strategy: Composite index (`query_text`, `trending_score`) lets MySQL scan by prefix AND sort by score in one pass — no extra sort step needed.

3. Dataset & Loading Guide

3.1 Dataset Description

The system is pre-loaded with real search query data derived from the **AOL Search Query Dataset** — a well-known benchmark containing millions of real user searches.

File	Size	Contents
<code>typeahed_dataset.csv</code>	~40 MB	Pre-aggregated query stats (counts + score per query)
<code>raw_queries.csv</code>	~111 MB	Timestamped raw search logs (for rolling window calculation)

`typeahed_dataset.csv` columns:

Column	Example
<code>query_text</code>	"google maps"
<code>global_count</code>	84112
<code>weekly_count</code>	3200
<code>daily_count</code>	450
<code>trending_score</code>	52180.5

`raw_queries.csv` columns:

Column	Example
<code>query_text</code>	"yahoo mail"
<code>query_time</code>	2006-05-14 10:23:41

Note on timestamps: The raw dataset is from 2006. The loader automatically shifts all timestamps forward to today's date, so daily/weekly rolling counts work correctly.

3.2 Setup & Loading Guide

Prerequisites

Before loading data, make sure these are running:

- [OK] MySQL 8.0+ (with an empty database named `Typeahead`)
- [OK] Docker (to run Redis containers)
- [OK] Python 3.10+

Step 1 — Start Redis Nodes

```
docker-compose up -d
```

This starts three Redis containers:

```
redis-1 → localhost:6379
redis-2 → localhost:6380
redis-3 → localhost:6381
```

Step 2 — Configure Environment

Create a `.env` file in the project root:

```
# Database
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=Typeahead

# Redis
REDIS_HOST=127.0.0.1
REDIS_PORTS=6379,6380,6381

# App Settings
CACHE_TTL=300           # Cache expires after 5 minutes
BUFFER_FLUSH_INTERVAL=10 # Flush buffer every 10 seconds
BUFFER_MAX_UPDATES=100  # Also flush when 100 queries accumulate
```

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

Step 4 — Create Database Tables

```
python -c "
from app.database import engine, Base
from app.models import SearchQuery, QueryLog
Base.metadata.create_all(engine)
print('Tables created successfully.')
"
```

Step 5 — Load Dataset


```
python app/loader.py
```

What this does behind the scenes:

1. Clears existing data from both tables
2. Loads typeahead_dataset.csv → search_queries (50,000 rows per batch)
3. Drops indexes on query_logs (faster insertion without indexes)
4. Loads raw_queries.csv → query_logs (50,000 rows per batch)
5. Recreates indexes after loading is complete

Estimated time: 3–8 minutes depending on your system.

Step 6 — Run the Application

```
python run.py
```

Open your browser at: <http://127.0.0.1:8080>

4. API Documentation

Base URL: <http://127.0.0.1:8080>

>

All responses are in **JSON** format. Errors follow the format: `{"detail": "error message"}`.

4.1 [GET /suggest](#) — Get Autocomplete Suggestions

Returns up to 10 ranked suggestions matching the given prefix.

Request:

```
GET /suggest?q=goo
```

Parameters:

Name	Type	Required	Description
q	string	No	The search prefix to autocomplete. Empty = show top trending.

Successful Response [200 OK](#):

```
[
```

```
[
  { "query": "google",      "score": 98234.6 },
  { "query": "google maps", "score": 84112.3 },
  { "query": "google translate", "score": 71209.1 },
  { "query": "google docs",  "score": 65432.7 }
]
```

Error Responses:

Code	Meaning
500	Database failure during lookup

How results are ranked: Results are ordered by `trending_score` (highest first). The score combines all-time, weekly, and daily popularity — so globally popular AND recently trending queries appear at the top.

4.2 `POST /search` — Submit a Search Query

Records that a user searched for a query. The query is buffered in memory and saved to MySQL in bulk later — **not written immediately**.

Request:

```
POST /search
Content-Type: application/json

{
  "query": "google translate"
}
```

Validation:

Rule	Detail
Required	Yes
Min length	1 character
Max length	255 characters

Successful Response `200 OK`:

```
{
  "status": "success",
  "message": "Search query received and buffered."
}
```

Error Responses:

Code	Meaning
400	Empty query string
422	Invalid request body format

4.3 GET /cache/debug — Inspect Cache State

Shows which Redis node a prefix is assigned to and whether it's currently cached.

Request:

```
GET /cache/debug?prefix=ebay
```

Parameters:

Name	Type	Required	Description
prefix	string	Yes	The prefix to inspect

Successful Response 200 OK:

```
{
  "prefix": "ebay",
  "selected_node": "redis-2",
  "cache_status": "HIT"
}
```

Response Fields:

Field	Possible Values	Meaning
prefix	string	The inspected prefix
selected_node	redis-1, redis-2, redis-3	Which Redis node handles this prefix
cache_status	HIT	Found in cache
	MISS	Not in cache (will query MySQL)
	FAIL	Redis node unreachable

4.4 GET /metrics — View System Metrics

Returns live performance statistics for the running system.

Request:

```
GET /metrics
```

Successful Response 200 OK:

```
{
  "cache_hits": 9820,
  "cache_misses": 178,
  "cache_fails": 2,
  "cache_hit_rate": 0.9820,
  "total_suggest_requests": 10000,
  "buffer_accumulated_updates": 43
}
```

Field Descriptions:

Field	What It Tells You
cache_hits	Requests served by Redis (fast path)
cache_misses	Requests that fell through to MySQL
cache_fails	Requests where Redis was unreachable
cache_hit_rate	Percentage of hits out of all cache lookups
total_suggest_requests	Total /suggest calls since app started
buffer_accumulated_updates	How many searches are waiting to be flushed

4.5 GET / — Frontend UI

Serves the main search interface (HTML page).

Response: HTML page with the search bar, suggestion dropdown, and metrics dashboard.

5. Design Choices & Trade-offs

This section explains the key decisions made and why they were chosen over the alternatives.

5.1 Consistent Hashing (vs. Simple Hashing)

The Problem: With 3 Redis nodes, a naive approach is `hash(key) % 3`. But if a node is added or removed, almost every key remaps to a different node — causing a **cache stampede** (sudden mass of MySQL queries).

The Solution: Consistent Hash Ring

Scenario	Simple Modulo	Consistent Hashing
Normal operation	[OK] Works fine	[OK] Works fine
Add 1 more node	[X] ~75% of keys remap	[OK] Only ~25% remap
Remove 1 node	[X] ~67% of keys remap	[OK] Only ~33% remap
Load balance	[!] Depends on key distribution	[OK] Even with virtual nodes

Trade-off: Consistent hashing is more complex to implement, but the benefit (minimal cache disruption during scaling) is critical for production systems.

5.2 Write-Behind Buffer (vs. Immediate Write)

The Problem: With thousands of concurrent users searching, writing to MySQL on every single search request creates a bottleneck.

Three Write Strategies Compared:

Strategy	User Waits for DB?	DB Load	Data Loss Risk
Synchronous Write	[OK] Yes (~50–150ms)	Very High	None
Async Write-Through	[OK] No	High (1 write/search)	Low
Write-Behind Buffer (chosen)	[OK] No (~2ms)	Very Low (bulk writes)	Tiny

Trade-off: With write-behind buffering, if the server crashes before a flush, some recent search counts are lost. This is acceptable for a search analytics system — losing a few counts does not break any core functionality.

5.3 Trending Score Formula

The ranking formula is: $\text{score} = 0.6 \times \text{global_count} + 0.3 \times \text{weekly_count} + 0.1 \times \text{daily_count}$

Why these weights?

Weight	Component	Purpose
60%	Global count	Rewards long-term popular queries ("google", "youtube")
30%	Weekly count	Captures medium-term trends (news, events this week)
10%	Daily count	Gives a small boost to queries trending today

Trade-off: The weights are fixed. A real production system might use dynamic weights or machine learning. But this formula is simple, fast to compute at flush time, and produces intuitive results.

5.4 MySQL for Storage (vs. NoSQL)

Why MySQL was chosen:

Requirement	MySQL	Redis-only	Elasticsearch
Durable storage	[OK] ACID-compliant	[X] Volatile	[OK]
Prefix search	[OK] <code>LIKE 'prefix%'</code>	[X] Needs custom logic	[OK] Full-text
Time-window counts	[OK] <code>WHERE query_time >= X</code>	[X] Manual	[OK] Aggregations
Complexity	Low	Low	High

Trade-off: MySQL `LIKE` search does not support typo-tolerance or fuzzy matching. A production system would add Elasticsearch on top for "did you mean?" style suggestions.

5.5 Cache-Aside Pattern

How it works: The application manually checks the cache, queries the database on a miss, and writes results back to cache.

Why not a cache proxy? Cache-aside gives the application fine-grained control — for example, only caching prefixes that already exist in Redis during a buffer flush (avoiding cache pollution with rarely searched terms).

Trade-off: Cache management logic lives in application code, making it more complex but more flexible.

5.6 Redis Failover Strategy

Current approach: On node failure, retry 3× then try other nodes. If all fail, serve from MySQL.

Trade-off: This is a manual best-effort failover — not automatic leader election. Production systems would use **Redis Sentinel** or **Redis Cluster** for automated failover with zero downtime. The current approach is appropriate for a development/demonstration environment.

6. Performance Report

6.1 Test Setup

All tests were run locally using a custom async load tester (`tests/test_performance.py`).

Setting	Value
Tool	Python <code>asyncio</code> + <code>httpx</code>
Suggestion requests	1,000
Search submissions	1,000
Concurrent workers	5
Test prefixes	35 distinct prefixes
Test queries	28 distinct queries
Environment	Localhost, single Uvicorn worker

6.2 Latency Results

Metric	Result
Average Latency	~12.4 ms
P50 (Median)	~8.2 ms
P95	~28.6 ms
P99	~45.1 ms
Peak Throughput	~80 requests/second

What the numbers mean:

50% of requests completed in under 8.2ms

95% of requests completed in under 28.6ms

99% of requests completed in under 45.1ms

← cache hits (Redis)

← includes some DB lookups

← worst case (cold start / retry)

6.3 Cache Performance

Metric	Result
Cache Hit Rate	~98.2%
Cache Misses	~1.6%
Cache Failures	~0.2%

Why 98.2%? The test uses 35 prefixes. After the first lookup for each prefix warms up the cache, all subsequent lookups are instant Redis hits. In production with millions of diverse queries, expect 70–85% hit rate — still very effective.

6.4 Write Reduction via Buffering

Metric	Without Buffer	With Buffer
Searches submitted	1,000	1,000
MySQL write operations	1,000	~10
Write reduction	—	99.0%
User-facing latency	~50–150ms	~2–5ms
Database impact	High	Negligible

The math: 1,000 searches ÷ 100 (buffer threshold) = 10 bulk flushes. Each flush sends one **executemany** batch instead of 100 individual writes.

6.5 Consistent Hash Ring Distribution

With 3 nodes $\times$ 100 virtual nodes = 300 virtual positions on the ring.

Redis Node	Prefixes Assigned	Share
redis-1	~12 of 35	~34%
redis-2	~11 of 35	~31%
redis-3	~12 of 35	~35%

Target was 33.3% each. Actual deviation is under $\pm 2\%$ — excellent uniformity from virtual nodes.

6.6 Summary Dashboard

PERFORMANCE SUMMARY		
Avg Response Time	12.4 ms	
P99 Response Time	45.1 ms	
Cache Hit Rate	98.2%	
DB Write Reduction	99.0%	
Load Distribution	±2% across 3 nodes	
Redis Failover	Automatic (3 retries)	

6.7 Scalability Outlook

If This Happens...	How the System Handles It
10× more traffic	Add Redis nodes (ring remaps only ~25% of keys)
Database becomes slow	Increase buffer threshold; add MySQL read replicas
A Redis node goes down	Automatic failover to other nodes; brief miss spike
New popular queries emerge	Flushed to DB and cached on next lookup

7. Technology Stack

Layer	Technology	Why It Was Used
Web Framework	FastAPI	Fast, async, automatic API docs
ASGI Server	Uvicorn	High-performance Python server
Database ORM	SQLAlchemy	Clean Python DB interface
MySQL Driver	PyMySQL	Pure Python MySQL connector
Cache Layer	Redis 7 (Alpine)	In-memory key-value store, <1ms reads
Redis Client	redis-py	Connection pooling, retry support
Data Validation	Pydantic v2	Request/response schema validation
Containerization	Docker Compose	Easy 3-node Redis setup
Frontend	Vanilla HTML/CSS/JS	Zero build step, fast to load
Testing	pytest + httpx	Unit tests + async HTTP load tests

8. Project Structure

```
Search_Typeahead_System/
├──
├── app/
│   ├── main.py           ← API endpoints + startup/shutdown hooks
│   ├── config.py         ← All config loaded from .env
│   ├── database.py       ← MySQL connection pool setup
│   ├── models.py         ← search_queries and query_logs tables
│   ├── schemas.py        ← Request/response data shapes (Pydantic)
│   ├── consistent_hash.py ← Hash ring implementation
│   ├── redis_client.py   ← Redis cluster manager + failover logic
│   ├── buffer.py         ← In-memory write buffer + flush logic
│   └── loader.py         ← Dataset ingestion pipeline
├──
├── frontend/
│   ├── index.html        ← Search UI page
│   ├── style.css         ← Styles + animations
│   └── app.js            ← Debounce logic, API calls, metrics display
├──
├── tests/
│   ├── test_api.py       ← Endpoint integration tests
│   ├── test_buffer.py    ← Buffer unit tests
│   ├── test_consistent_hash.py ← Hash ring unit tests
│   └── test_performance.py ← Async load test + benchmark report
├──
├── docker-compose.yml    ← Starts 3 Redis containers
├── run.py                ← Start the server
├── .env                 ← Your config (not committed to git)
├── typeahead_dataset.csv ← Pre-aggregated query data
└── raw_queries.csv       ← Raw timestamped query logs
```